MixBytes()

[SECURITY AUDIT REPORT]

Lido CMv2 Top-Up Control

PREPARED FOR LIDO

REPORT DATE

November 12, 2026

[CONTENTS]

Table of Contents

1. Introduction 3
   - 1.1 Disclaimer 3
   - 1.2 Executive Summary 3
   - 1.3 Project Overview 5
   - 1.4 Security Audit Methodology 9
   - 1.5 Risk Classification 11
   - 1.6 Summary of Findings 12
2. Findings Report 12
   - 2.1 Critical 12
   - 2.2 High 12
   - 2.3 Medium 12
   - 2.4 Low 12
3. About MixBytes 13

1.1 Disclaimer

The audit makes no statements or warranties regarding the utility, safety, or security of the code, the suitability of the business model, investment advice, endorsement of the platform or its products, the regulatory regime for the business model, or any other claims about the fitness of the contracts for a particular purpose or their bug-free status.

1.2 Executive Summary

Lido Curated Module v2 allocates Lido's buffered ETH between curated node operators using registry weights, tracked internal and external stake, and operator capacity. The reviewed change adds per-operator top-up enablement flags and weighted allocation for topping up existing 0x02 validators, while retaining the separate initial-deposit path. Staking Router supplies deposit budgets and key-level limits; MetaRegistry supplies allocation weights and external stake.

The interim security review was conducted over 2 working days via manual code review and a proprietary AI-assisted analysis tool.

The review covered `TopUpAllocationLib.sol` in full and the target-commit diffs of the other scoped files against previous audit commit `31e9f1f4ea410b541293648c7bfb7dbed9b372bb`, together with security-relevant interactions with Staking Router and MetaRegistry. In addition to the standard checks for access control, reentrancy, arithmetic safety, rounding, storage layout, and upgrade initialization, the following project-specific attack vectors were investigated:

- **Allocation manipulation.** Duplicate operators or keys, malicious request subsets, weight normalization, quantization, per-key caps, and global share-baseline integrity.
- **Policy and lifecycle transitions.** Operator enablement, disabled or slashed operators, initial-deposit isolation, pause behavior, and owner or role changes.
- **Upgrade and integration safety.** Proxy finalization, role assignments, nonce semantics, stale deposit information, and buffer utilization across modules.

Staking Router, MetaRegistry, Accounting, Verifier, deployment scripts, and the depositor bot were outside the line-by-line audit scope. Their code and configuration were examined only where needed to validate integration and privilege assumptions in the scoped contracts.

Below we set out our overall assessment, key assumptions, and main recommendations.

- **Overall assessment.** No findings were identified at any severity level.
- **Code structure.** The allocation logic is modular and documents its principal invariants and integration assumptions. No direct buffer theft or budget overdraw was identified.
- **Integration notice - module nonce.** Successful `allocateDeposits` calls update tracked balances without advancing the module nonce. On-chain state and the reviewed Router/depositor path are unaffected, but custom nonce-only consumers can remain stale until an unrelated nonce change.
- **Out-of-scope integration - buffer utilization.** In the pinned bot, `_phase_full_and_topup` calls `get_deposit_allocations(..., is_top_up=True)` once and stores the resulting amount in each `ModuleCandidate`. `CMv2TopUpStrategy.get_topup_candidates` then obtains Curated's operator allocation and filters keys using Beacon Chain eligibility, so it can return no candidates even when the Router assigned Curated a nonzero amount. `_top_up_to_module` converts that result to `PhaseOutcome.SKIPPED`, but the loop continues with the original per-module allocation vector and never recomputes it. Curated's unused share is therefore not redirected to later modules in that cycle; the ETH remains in Lido's buffer and stake deployment is delayed rather than funds being lost.

- **Out-of-scope integration - stale previews.** `CMv2TopUpStrategy.get_topup_candidates` calls `cmv2.get_deposits_allocation` before its internal exception handlers. If Curated deposit information is stale, this call reverts with `DepositInfoIsNotUpToDate`. The call to the strategy in `_top_up_to_module` has no per-candidate exception handler, so the exception escapes the candidate loop and later modules are not tried. The outer `Executor` catches the unexpected exception and retries on a later block, which can repeat the same starvation while Curated remains stale. The permissionless `batchDepositInfoUpdate` provides recovery; this is an integration liveness issue, not a daemon crash or direct fund-loss path.
- **Deployment assumption - atomic upgrade.** The v1-to-v2 proxy upgrade and `finalizeUpgradeV2` must remain atomic, as required by the interface, so the version-2 initializer cannot be called independently.
- **ENABLE_TOP_UP_ALLOCATION_ROLE.** The mainnet deployment configuration assigns this role to CMC. It can enable top-ups for an existing operator, but there is no corresponding CMC disable function. Restoration therefore requires committee action, while CMC cannot selectively stop an operator's top-ups through this flag.
- **Operator owner.** The `managerAddress` with extended permissions, otherwise the `rewardAddress`, can disable top-ups for its own operator, including while the module is paused. This enables local opt-out, but a compromised owner key can cause operator-level top-up liveness loss until CMC re-enables it.
- **STAKING_ROUTER_ROLE.** Staking Router executes initial deposits and top-ups and updates target limits. It cannot enable or disable the new per-operator flags.
- **DEFAULT_ADMIN_ROLE.** The Aragon Agent administers roles after deployment, including granting or revoking the enable role. It has no direct flag-disable entry point; stronger intervention requires governance or an upgrade.
- **PAUSE_ROLE and RESUME_ROLE.** CircuitBreaker and ResealManager hold broad emergency permissions, and CMC is the configured CircuitBreaker pauser. These permissions do not provide selective top-up control. The scoped `obtainDepositData` and `allocateDeposits` entry points do not check `whenResumed`.

This permission model sacrifices selective committee response in favor of limits on unilateral committee control. If an operator is unresponsive, CMC cannot disable only that operator's top-ups; if the owner key is compromised, the attacker can disable them and require CMC restoration. Disabling excludes the operator's weight, internal stake, and external stake from the executable top-up baseline and therefore changes other operators' targets. It does not disable initial deposits, so separate target and key controls remain necessary for a full wind-down.

1.3 Project Overview

Summary

CLIENT	Lido
CATEGORY	Liquid Staking
PROJECT	CMv2 Top-Up Control
TYPE	Solidity
PLATFORM	EVM
TIMELINE	29.10.2026 - 30.10.2026

Scope of Audit

`src/lib/TopUpAllocationLib.sol` `src/CuratedModule.sol` `src/interfaces/ICuratedModule.sol`

`src/lib/allocator/CuratedDepositAllocator.sol`

Versions Log

DATE	COMMIT HASH	NOTE
29.10.2026	31e9f1f4ea410b541293648c7bfb7dbed9b372bb	Diff Base
29.10.2026	7bdcfb631b13d24e8fae3b5ba0e11d4be07b84f9	Audited Commit

Mainnet Deployments

FILE	ADDRESS	BLOCKCHAIN
CuratedModule.sol (implementation)	0x02bEFd784c6C382a021774472563671Ae23980C0	Ethereum
CuratedDepositAllocator.sol	0x691e8BA3DBf3E2E33a8C5641B15d4d24F3837F37	Ethereum

Contract code and configuration checks in this section use Ethereum mainnet state at finalized block `26391733`. Deployment and funding transactions were additionally inspected in their respective historical blocks.

Deployment transactions. Both contracts were deployed from the audited commit `7bdcfb631b13d24e8fae3b5ba0e11d4be07b84f9`, as recorded in the client's deployment artifacts `deploy-mainnet.json` and `transactions-top-up-allocation.json` (staking-modules#918, commit `5b65958b1c67b41710c9e13869cffa745bc0235b`, whose only parent is the audited commit and which changes only these two files). The deployer is the EOA `0x96Afa46bDD33b5560Ba82DBc29019a430A626c63`. It created `CuratedDepositAllocator` through the deterministic `CREATE2` deployer `0x4e59b44847b379578588920cA78FbF26c0B4956C` with a zero salt in transaction `0xdce1809ef2f777952c73ff4f64fbfce0278db45534e749a626c641d6e71eebeb` (block `26385524`, nonce 0). It created `CuratedModule` with a plain `CREATE` in transaction `0x3022badc47c791617d9e6001bfc45e43b760191ffe90962a7c9a95faf3c3ec0f` (block `26385554`, nonce 1). We recomputed both addresses from the transaction data. The deployer's only other transaction (nonce 2, `0xa7c15f5093487737505f96d7c46a2b7d876fb43b5232d62c9b23efd6e66207a2`) returns the unused 0.0226 ETH, with empty calldata, to the Safe `0x7FEa69d107A77B5817379d1254cc80D9671E171b` that funded it.

Bytecode identity. We recompiled the audited commit with the official `solc 0.8.33` binary using the settings from the repository's `foundry.toml`: EVM version `osaka`, optimizer enabled with 150 runs, `bytecode_hash = "none"`, OpenZeppelin `5.4.0`. The executable runtime bytecode of both deployments matches this build. The only regions excluded from the comparison are constructor-assigned immutables, linked library addresses, and the library's own address in the `CuratedDepositAllocator` call guard. With `bytecode_hash = "none"`, the CBOR metadata contains only the compiler version, and it also matches byte for byte. The creation code in both transactions equals the compiled creation code. For `CuratedModule`, it is linked with the library addresses from the manifest and followed by the ABI-encoded constructor arguments. `TopUpAllocationLib` has only internal functions and is inlined into both contracts, so it needs no separate deployment. Both contracts are verified on Etherscan, and the 69 source files published for `CuratedModule` on Sourcify are byte-identical to the audited commit and OpenZeppelin `5.4.0`.

Constructor arguments and immutables. The constructor arguments decoded from the deployment transaction and the immutables read from the deployed code match the manifest:

- module type `curated-onchain-v2`;
- LidoLocator `0xC1d0b3DE6792Bf6b4b37EccdcC24e45978Cfd2Eb`;
- ParametersRegistry `0xffC1C5d59CeAC6F6c27E701F04a70cb50474607C`;
- Accounting `0x2F91e3A8C5d6593bf4F8403fCfeCcd62dF59f6F6`;

- ExitPenalties `0x004aFb7DAA7dEA20EbAaB75c9F4892C879FaCCe0`;
- MetaRegistry `0xA64b339eebD3dC3De848298B6a140955932901d8`.

The derived immutables also match: `STETH` is `0xae7ab96520DE3A18E5e111B5EaAb095312D7fE84`, equal to `LidoLocator.lido()`, and `FEE_DISTRIBUTOR` is `0x367d23c756599c20DCc8D6943F4976E8F88D60d7`, equal to `Accounting.FEE_DISTRIBUTOR()`. All eight immutables are identical to those of the currently active implementation `0x959fC67FE53c8A6C7a1AEd73430Aa07a36eD9337`.

Unchanged external libraries. `CuratedDepositAllocator` is the only external library that was redeployed. The other external libraries listed in the manifest were not changed:

LIBRARY	ADDRESS
`AssetRecovererLib.sol`	0x37aDa408AE3c3992953688e2CCb9eE7a3dfdA902
`GeneralPenaltyLib.sol (GeneralPenalty)`	0xF05545ED71c60bBba6E73B6B70B15D4f5F22C0f4
`NOAddresses.sol`	0x9D9c8799189c797f6e2dA74F71aDF84492adA7D3
`NodeOperatorOps.sol`	0xDD42EE5D54A1822021782F3F455bb99fBC19499A
`StakeTracker.sol`	0xbb6E4Db18182d45038F91B9F1195291c206fd8d2
`WithdrawnValidatorLib.sol`	0x3bf9674f062aF9BA94FdAe9Fcdf2D0001FFf0a3A
`BondCurvesLib.sol (used by Accounting, not linked into CuratedModule)`	0xC4511d09639e5E174506083443da230D39196323

- **Manifest.** The deployment updates only three entries of `deploy-mainnet.json`: `CuratedModuleImpl`, `ExternalLibraries.CuratedDepositAllocator`, and `git-ref`. The addresses of the libraries above did not change.
- **Linking.** Besides `CuratedDepositAllocator`, the new implementation links the six libraries above that `CuratedModule` uses (all except `BondCurvesLib`), at the same addresses as the active implementation `0x959fC67FE53c8A6C7a1AEd73430Aa07a36eD9337`. The only link that changed is `CuratedDepositAllocator`, which moved from `0xa4fCD4dDa0e4a847142E3592C97c77d8B9B3Cf5F` to `0x691e8BA3DBf3E2E33a8C5641B15d4d24F3837F37`.
- **Source.** Every source file that each of the seven libraries compiles from is identical between the original CMv2 deployment commit `3b0c016c77590c208f2b71ca51410cb55ea2dcc2` and the audited commit. Between these commits, `src/` changes only in `CuratedModule.sol`, `ICuratedModule.sol`, `IStakingRouter.sol`, `TopUpAllocationLib.sol`, and `CuratedDepositAllocator.sol`, and none of the seven libraries imports any of them.
- **Bytecode.** The on-chain runtime code at each of the seven addresses matches the audited-commit build, excluding only the library's own address in its call guard.

Initialization and upgrade state. The new implementation's constructor disabled initializers. The OpenZeppelin `Initializable` slot holds `type(uint64).max`, so neither `initialize` nor `finalizeUpgradeV2` (both `reinitializer(2)`) can be called on the implementation, and it has no role holders. At the snapshot block, the CuratedModule proxy `0xDa5F930cE326EB5205085D66c72A4E79d60cB8C1` still points to the previous implementation `0x959fC67FE53c8A6C7a1AEd73430Aa07a36eD9337`. Its admin and sole `DEFAULT_ADMIN_ROLE` holder is the Lido Aragon Agent `0x3e40D73EB977Dc6a537aF587D48316feE66E9C8c`, its initialized version is 1,

and no account holds `ENABLE_TOP_UP_ALLOCATION_ROLE` yet. The proxy upgrade and `finalizeUpgradeV2` are therefore still pending. As noted in the Executive Summary, they must be executed atomically.

1.4 Security Audit Methodology

Stage 1 Interim Audit

01 PROJECT ARCHITECTURE REVIEW

OBJECTIVE: UNDERSTAND THE OVERALL STRUCTURE OF THE PROTOCOL AND IDENTIFY POTENTIAL SECURITY RISKS, INCLUDING THE PRIMITIVES AND ABSTRACTIONS IMPLEMENTED BY THE SYSTEM AND WHERE DESIGN OR INTEGRATION WEAKNESSES COULD BE EXPLOITED.

- Understanding overall system design and contract interactions
- Mapping responsibilities of each component and protocol workflows
- Conducting a thorough line-by-line review of contracts and modules
- Tracing execution paths and mapping state transitions
- Identifying trust boundaries and external integrations
- Forming an independent architectural understanding directly from code before validating documentation
- Running the project test suite to observe behavior and encoded invariants

02 ADVERSARIAL CODE REVIEW

OBJECTIVE: IDENTIFY POTENTIAL VULNERABILITIES SPECIFIC TO THE PROTOCOL ARCHITECTURE, BUSINESS LOGIC, AND ECONOMIC MECHANISMS.

- Analyzing the codebase from an attacker's perspective: what can fail, not only what works
- Searching for edge cases, invalid assumptions, unexpected call sequences, and unsafe state transitions
- Attempting to violate protocol invariants and find where guarantees break
- Analyzing economic attack surfaces: oracle dependencies, share pricing, access control, cross-contract interactions
- Writing targeted tests, modelling adversarial scenarios, mutation testing, fuzzing, and PoC exploits
- Independent exploit-path discovery by each auditor to reduce anchoring bias
- Collaborative pair-auditing of high-risk code led by the Team Lead

03 SYSTEMATIC VULNERABILITY ANALYSIS

OBJECTIVE: APPLY STRUCTURED ANALYSIS AND KNOWN ATTACK PATTERNS TO ENSURE COMPREHENSIVE COVERAGE ACROSS THE CODEBASE.

- Reviewing code against an internally maintained vulnerability checklist derived from past exploits and research
- Analyzing common DeFi attack classes: reentrancy, accounting manipulation, oracle manipulation, privilege escalation, state desynchronization
- Using proprietary AI tooling to surface candidate issues across broader attack vectors
- Manually validating and triaging AI-generated candidates

04 CONSOLIDATION OF AUDITORS' REPORTS

OBJECTIVE: MERGE INTERIM INPUTS FROM ALL AUDITORS INTO A COHERENT REPORT WITH CONSISTENT SEVERITY LEVELS AND REPRODUCIBLE FINDINGS.

- Cross-checking findings across auditors; consolidating and deduplicating issues
- Resolving discrepancies in severity assessments
- Using proprietary AI tooling to assist with report drafting and consistency checks
- Manually reviewing and finalizing the report narrative for client review

Stage 2 Re-audit & Mainnet Verification

RE-AUDIT

OBJECTIVE: VERIFY THAT CLIENT REMEDIATIONS ARE CORRECTLY IMPLEMENTED AND THE UPDATED CODE INTRODUCES NO NEW VULNERABILITIES.

- Confirming each remediation matches the original recommendation
- Documenting rationale for any unresolved findings
- Verifying modified logic and integration points remain secure
- Executing the test suite after remediation, targeting fixed areas
- Running AI tooling on the updated codebase

MAINNET DEPLOYMENT VERIFICATION

OBJECTIVE: FINAL VERIFICATION OF DEPLOYED CONTRACTS AND CONFIGURATION BEFORE ISSUING THE PUBLIC REPORT.

- Verifying deployed bytecode against audited source across networks
- Matching bytecode to the build from the audited commit, identical compiler settings
- Reviewing constructor and initializer arguments
- Verifying proxy order, initialization flow, and admin configuration
- Ensuring implementations aren't left uninitialized or exposed to init front-running

1.5 Risk Classification

Severity Level Matrix

SEVERITY	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
LIKELIHOOD: HIGH	CRITICAL	HIGH	MEDIUM
LIKELIHOOD: MEDIUM	HIGH	MEDIUM	LOW
LIKELIHOOD: LOW	MEDIUM	LOW	LOW

IMPACT

HIGH - Theft exceeding 0.5% of the protocol's TVL, partial or complete blocking of funds on the contract without the possibility of withdrawal (>0.5%), or loss of user funds exceeding 1% for users interacting with the protocol.

MEDIUM - Contract lock that can only be resolved through a contract upgrade, one-time theft of rewards or an amount up to 0.5% of the protocol's TVL, or funds lock where withdrawal is still possible by an administrator.

LOW - One-time contract lock that can be resolved by an administrator without requiring a contract upgrade.

LIKELIHOOD

HIGH - An event with an estimated 50–60% probability of occurring within a year, which can be triggered by any actor (e.g. due to a market condition that the actor cannot influence).

MEDIUM - An unlikely event (10–20% probability of occurring) that can be triggered by a trusted actor.

LOW - A highly unlikely event that can only be triggered by the contract owner.

ACTION REQUIRED

CRITICAL - Must be fixed as soon as possible.

HIGH - Strongly advised to be fixed in order to minimize potential risks.

MEDIUM - Recommended to be fixed to improve security and stability.

LOW - Recommended to be fixed to improve overall robustness and efficiency.

FINDING STATUS

● FIXED - The recommended fixes have been implemented and the issue no longer impacts the security of the protocol.

● PARTIALLY FIXED - The fixes have been partially implemented, reducing the impact, but the issue has not been fully resolved.

● ACKNOWLEDGED - The fixes were not implemented; the finding is unresolved or was accepted by the team without code changes.

[1 INTRODUCTION]

1.6 Summary of Findings

Findings Count

0	0	0	0	0
CRITICAL	HIGH	MEDIUM	LOW	TOTAL

Findings Statuses

All finding statuses reflect the state of the code at the latest revision listed in the Versions Log.

[2 FINDINGS REPORT]

2.1 Critical

NO CRITICAL SEVERITY FINDINGS

2.2 High

NO HIGH SEVERITY FINDINGS

2.3 Medium

NO MEDIUM SEVERITY FINDINGS

2.4 Low

NO LOW SEVERITY FINDINGS

[3 ABOUT MIXBYTES]

Built on Trust

MixBytes is a blockchain security firm specializing in the analysis of decentralized protocols and smart contract systems. The company helps Web3 teams build resilient protocol architecture, smart contract logic, and economic mechanisms through a combination of AI-assisted analysis and senior human expertise.

Rather than focusing solely on one-time audits before deployment, MixBytes works with protocols across their entire lifecycle - from early architecture design to production audits and ongoing protocol evolution. Our team consists of experienced security researchers, engineers, and protocol analysts with deep expertise in DeFi systems, adversarial protocol analysis, and smart contract security.

Over the years, MixBytes has worked with many of the most widely used protocols in the ecosystem, including Lido, Aave, Curve, 1inch, OKX, Mantle, Fluid, Gearbox, Resolv, and others. To enhance analysis efficiency and coverage, MixBytes also develops and uses internal AI-assisted tooling that helps surface potential risk signals during development and code review.

SECURED TVL	AUDITS DELIVERED	WEB3 PROTOCOLS
$50B+	300+	80+

Protocol Security Lifecycle

MixBytes supports protocols across the full lifecycle of their development and operation.

Web-3 Design Review

Independent assessment of protocol architecture and economic design before critical decisions are embedded in production code.

AI Tooling

AI-assisted analysis integrated into development workflows to surface potential risk signals during protocol development.

Smart Contract Audit

Comprehensive manual verification of smart contract logic and protocol invariants before production deployment.

Security Support

Continuous expert support for protocol upgrades, integrations, governance changes, and evolving attack surfaces after launch.

CONTACTS

hello@mixbytes.io